Hong Kong Stock Price Prediction Analysis Report

Report Information

Project Title: Hong Kong Stock Market Trend Prediction Analysis
Subject Area: Prediction Analytics
Report Generated: March 2026
Report Version: v1.0

Table of Contents

1. Cover Page
2. Introduction
3. Data Source and Transformation
4. Analysis Steps and Trials
5. Model Evaluation
6. Prediction and Conclusions

1. Cover Page

Project Title

Hong Kong Stock Market Trend Prediction Analysis

Group Members

(Individual Project - Single Group)

Student ID	Name	Role
25127457G	Chen Haonan	Project Lead & Data Analyst

Instructor: (To be completed)
Completion Date: March 2026

---

2. Introduction

2.1 Project Background and Significance

Stock markets, as indicators of economic health, reflect market participants' expectations about future economic prospects. Hong Kong, as an international financial center, has a stock market characterized by high liquidity and diversified investment portfolios.

Core Significance of the Project: - Establish quantitative prediction models through historical data analysis - Compare the effectiveness of different forecasting methods - Provide data support for investment decisions - Validate the feasibility of machine learning applications in finance

2.2 Project Objectives

Primary Objectives: 1. Collect and preprocess historical data for Hong Kong stocks 2. Establish multiple prediction models (Linear Regression, Decision Tree, LSTM) 3. Analyze and compare the prediction performance of different models 4. Identify the optimal forecasting approach 5. Provide actionable recommendations

Research Questions: - Which model is most suitable for stock price prediction? - How do prediction difficulties vary across different stocks? - What are the main factors affecting model performance?

2.3 Report Structure

This report follows a standard data analysis workflow: data collection → preprocessing → model development → evaluation → conclusions, systematically presenting each project phase.

---

3. Data Source and Transformation

3.1 Data Source

Stock Selection: Listed companies on the Hong Kong Exchanges and Clearing Limited (HKEX)

Stock Code	Company Name	Industry	Date Range	Total Records
2318.HK	Ping An Insurance	Finance	2017/10/27 - 2025/10/23	1967
3690.HK	Meituan	Internet	2017/10/27 - 2025/10/23	1967
0288.HK	Golden Bull Limited	Small Cap	2017/10/27 - 2025/10/23	1967
0002.HK	CLP Holdings	Utilities	2017/10/27 - 2025/10/23	1967
0005.HK	HSBC Holdings	Finance	2017/10/27 - 2025/10/23	1967

Data Characteristics: - **Time Range**: October 27, 2017 to October 23, 2025 (~8 years of historical data) - **Data Frequency**: Daily close - **Data Features**: OHLCV (Open, High, Low, Close, Volume)

3.2 Key Data Description

Data Column Definitions

Column	Description	Data Type
Date	Trading date	Date
Open	Opening price	Float
High	Highest price	Float
Low	Lowest price	Float
Close	Closing price	Float
Volume	Trading volume	Integer
MA10	10-day moving average	Float
MA20	20-day moving average	Float

Prediction Target

- **Prediction Object**: Closing Price
- **Prediction Period**: 1 day ahead
- **Prediction Method**: Based on past 30 days of data to predict the 31st day's closing price

3.3 ETL Process

3.3.1 Extract

```
Source Data → CSV Loading → Pandas DataFrame
```

- Load local CSV files using `pandas.read_csv()`
- Data validation: verify column completeness and data types

3.3.2 Transform

Step 1: Data Cleaning - Handle missing values: fill MA with closing price - Remove outliers: identify and handle abnormal fluctuations - Date conversion: ensure uniform date format

Step 2: Feature Engineering - Calculate moving averages (MA10, MA20) - Generate 30-day sliding window feature sequences - Feature normalization: MinMaxScaler scale data to [0, 1]

Step 3: Data Splitting

```
Raw Dataset
├─ Training Set (80%)
└─ Test Set (20%)
```

- Split by time order (prevent data leakage)
- Training set: for model learning
- Test set: for model evaluation

3.3.3 Load

```
Preprocessed Data → Input to Models → Generate Predictions
```

- Convert to NumPy array format
- Maintain original indices for result association

3.4 Data Statistics Summary

Stock Data Overview

Stock	Records	Close Range	Avg Price	Volatility	Date Range
0288.HK	1967	2.98-8.56	5.12	Moderate	2017/10/27 - 2025/10/23
0002.HK	1967	28.50-102.50	56.34	High	2017/10/27 - 2025/10/23
2318.HK	1967	3.50-25.80	14.62	High	2017/10/27 - 2025/10/23
0005.HK	1967	20.96-110.80	58.93	**Very High**	2017/10/27 - 2025/10/23
3690.HK	1967	32.50-235.40	98.76	Very High	2017/10/27 - 2025/10/23

Key Observations: - All stocks have complete data from October 27, 2017 to October 23, 2025 - 8-year historical period covering multiple market cycles - All stocks have complete data with no severe missing values - 0005.HK shows significant price fluctuations (max is 5x min) - Different stocks have vastly different price ranges requiring separate modeling

---

4. Analysis Steps and Trials

4.1 Overall Analysis Framework

```
Data Loading
  ↓
Data Preprocessing
  ↓
Feature Selection (select optimal features for each stock)
  ↓
┌──────────────────────────────────┐
│ Train Three Models in Parallel:  │
│ • Linear Regression (LR)         │
│ • Decision Tree (DT)             │
│ • LSTM                           │
└──────────────────────────────────┘
  ↓
Model Evaluation and Comparison
  ↓
Conclusions and Recommendations
```

4.2 Feature Selection

4.2.1 Available Features

- **Price Features**: Close, Open, High, Low
- **Volume**: Volume
- **Derived Features**: 10-day MA, 20-day MA

4.2.2 Feature Selection Strategy

The algorithm uses **sequential search**:

1. **Round 1**: Single feature testing
   - Test each feature individually
   - Evaluate using fast linear regression
2. **Round 2**: Two-feature combinations
   - Test Close + other feature pairs
   - Assess feature interaction effects
3. **Round 3**: Three-feature combinations
   - Test most promising three-feature combinations
4. **Feature Ranking**:
   - Sort by RMSE from small to large
   - Select top 3 best combinations

4.2.3 Optimal Features by Stock

Stock	Optimal Features	Num Features
0288.HK	Close	1
0002.HK	Close, Open, High	3
2318.HK	Close, High	2
0005.HK	Close, MA10	2
3690.HK	Close, Volume, MA10	3

Key Findings: - Close price is essential for all models - Optimal feature combinations vary across stocks - Decision trees prefer single features (avoid overfitting)

4.3 Model Implementation Details

4.3.1 Linear Regression Model

Model Theory: $$\hat{y} = \beta0 + \beta1 x1 + \beta2 x2 + ... + \betan x_n$$

Implementation: - Use scikit-learn's `LinearRegression` class - Input: 30-day historical feature vector - Output: predicted closing price for next day

Configuration: The model uses default scikit-learn parameters with fit_intercept=True for optimal results.

Advantages: - Fast computation (<1 second for 5 stocks) - Highly interpretable - Supports linear extrapolation

Disadvantages: - Assumes linear relationship - Cannot capture complex nonlinear patterns

4.3.2 Decision Tree Model

Model Theory: - Recursively partition feature space - Each leaf node represents a predicted value

Implementation: - Use scikit-learn's `DecisionTreeRegressor` class - Based on CART (Classification And Regression Trees) algorithm

Optimized Configuration: - Maximum tree depth: 6 (reduced from 10 for better generalization) - Minimum samples to split: 15 (increased from 5 to prevent overfitting) - Minimum samples in leaf: 5 (newly added to improve stability) - Random state: 42 for reproducibility

Optimization History:

Stage	max_depth	minsamplessplit	Result
Initial	10	5	Overfitting
Improved	6	15	✓ Better

Improvement Results: - 0002.HK: RMSE from 0.7151 → 0.5984 (↓16.3%) - 0288.HK: RMSE from 0.9284 → 0.8773 (↓5.5%) - 3690.HK: RMSE from 8.4527 → 6.2861 (↓25.6%)

Known Limitations:

- **Data Drift Issue (0005.HK)**
  - Training range: [20.96, 56.74]
  - Test range: [53.50, 110.80]
  - Test max is 95% higher than training max
  - **Result**: Constant predictions (cannot extrapolate)

Solutions Implemented: - Use only Close feature (avoid high-dimensional overfitting) - Reduce tree depth (limit model complexity) - Increase minimum split samples (improve generalization)

4.3.3 LSTM Model

Model Theory: - Recurrent neural network variant - Designed for time series learning - Uses gate mechanisms to control information flow

Network Architecture:

```
Input Layer (30 time steps)
  ↓
LSTM Layer 1 (100 units, activation='relu')
  ↓
Dropout (0.1)
  ↓
LSTM Layer 2 (50 units, activation='relu')
  ↓
Dropout (0.1)
  ↓
Dense Layer (25 units, activation='relu')
  ↓
Output Layer (1 unit)
```

Configuration: The LSTM network is implemented with the following architecture: - First LSTM layer: 100 units with ReLU activation and return sequences enabled - Dropout layer: 0.1 to prevent overfitting - Second LSTM layer: 50 units with ReLU activation - Additional dropout layer: 0.1 for regularization - Dense layer: 25 units with ReLU activation - Output layer: 1 unit for single price prediction

Training parameters: - Optimizer: Adam - Loss function: Mean Squared Error (MSE) - Epochs: 100 - Batch size: 32 - Verbose: No logging during training

Key Innovation: Data Normalization

Status	Configuration	RMSE (2318.HK)	Improvement
Without Normalization	Raw Data	14.62	-
With Normalization	**MinMaxScaler**	**1.64**	**□ 88.8%**

Normalization Process:

```
Raw Data → MinMaxScaler(0-1)
  ↓ Fit on training set
  ↓ Transform test set
  ↓ Train model
  ↓ Inverse transform predictions
```

Advantages: - Learn nonlinear time series patterns - Significant performance improvement after normalization - Handle data outside training range

Disadvantages: - Long training time (4-8 minutes for 5 stocks) - Requires sufficient data samples - Complex hyperparameter tuning

4.4 Experiments and Iterations

Experiment 1: LSTM Non-Normalization Problem

Discovery: - LSTM predictions consistently lower than actual prices - 2318.HK: Predicted 29.91, Actual 43.82 (32% underestimation)

Diagnosis: - Raw stock price range: 20-110 - Model without normalization exhibits vanishing gradient

Solution: - Apply MinMaxScaler normalization to [0, 1] - Fit on training data before training, inverse transform after prediction

Result: 88.8% RMSE improvement ✓

Experiment 2: Decision Tree Data Drift Problem

Discovery: - 0005.HK Decision Tree predicts constant: 56.36 - All 358 test samples have identical prediction

Root Cause Analysis:

```
┌─ Decision Tree is "piecewise constant" model
│   ↓
│   Cannot extrapolate beyond training range
│   ↓
│   All values outside range map to same leaf node
└─ Result: Constant prediction
```

Data Comparison:

```
Training: [20.96, ......, 56.74]
Test:     [53.50, ......, 110.80]
                        ↑
                   95% out of range
```

Solutions Attempted: 1. □ Adjust maxdepth → Still fails 2. □ Increase minsamples_split → Still fails 3. ✓ **Disable multi-feature** (use Close only) → Partial improvement 4. ✓ **Reduce tree depth to 6** → Minor improvement

Final Conclusion: - This is a data characteristic problem, not a parameter problem - Decision Tree not recommended for 0005.HK - Other 4 stocks perform well

4.5 Model Implementation Summary

The complete implementation includes: - **Data pipeline**: Automated loading, cleaning, and feature engineering - **Feature selection**:

Systematic evaluation of feature combinations per stock - **Model training**: Parallel training of three models with optimized parameters - **Result evaluation**: Comprehensive performance metrics and error analysis - **Output generation**: CSV predictions and visualization charts

All implementations follow best practices for time series analysis: - Maintaining temporal order in train-test split (no shuffling) - Individual scaler fitting on training data only - Prevention of data leakage through proper sequence handling - Reproducible results with random seeds

---

5. Model Evaluation

5.1 Evaluation Metrics

5.1.1 Metric Definitions

RMSE (Root Mean Squared Error) $$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

- Same unit as target variable (price)
- More sensitive to large errors
- **Primary evaluation metric**

MSE (Mean Squared Error) $$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MAE (Mean Absolute Error) $$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

MAPE (Mean Absolute Percentage Error) $$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}i}{yi}\right| \times 100\%$$

5.2 Model Performance Comparison

5.2.1 Complete Performance Table

Rank	Stock	Model	MSE	RMSE	MAE	MAPE
1	0288.HK	LR	0.0113	**0.106**	0.089	1.75%
2	0288.HK	LSTM	0.0718	0.268	0.211	4.20%
3	0288.HK	DT	0.7699	0.877	0.652	10.23%
4	0002.HK	LR	0.2938	**0.542**	0.421	0.98%
5	0002.HK	DT	0.3581	0.598	0.487	1.15%
6	0002.HK	LSTM	2.2411	1.497	1.156	2.66%
7	2318.HK	LR	1.2634	**1.124**	0.893	4.52%
8	2318.HK	DT	1.5648	1.251	0.998	5.04%
9	2318.HK	LSTM	2.9652	1.722	1.367	6.92%
10	0005.HK	LR	1.4136	**1.189**	0.934	1.55%
11	0005.HK	LSTM	54.5525	7.386	5.876	9.78%
12	0005.HK	DT	758.0206	27.514	21.875	36.45%
13	3690.HK	LR	29.8878	**5.467**	4.234	2.87%
14	3690.HK	LSTM	38.5212	6.206	4.876	3.29%
15	3690.HK	DT	39.5143	6.286	4.923	3.32%

5.2.2 Model Summary Statistics

Average Performance by Model:

Model	Avg RMSE	Ranking	Speed	Stability
Linear Regression	1.666	□ **1st**	⚡⚡⚡	□□□□□
LSTM	3.356	□ **2nd**	⚡	□□□□
Decision Tree	7.105	□ **3rd**	⚡⚡	□□□

Top 5 Best Configurations: 1. 0288.HK + Linear Regression (RMSE: 0.106) 2. 0288.HK + LSTM (RMSE: 0.268) 3. 0002.HK + Linear Regression (RMSE: 0.542) 4. 0002.HK + Decision Tree (RMSE: 0.598) 5. 2318.HK + Linear Regression (RMSE: 1.124)

5.3 Model Comparison Analysis

5.3.1 Best Model by Stock

Stock	Best Model	RMSE	2nd Best	Avoid
0288.HK	**Linear Regression**	0.106	LSTM	DT
0002.HK	**Linear Regression**	0.542	Decision Tree	LSTM
2318.HK	**Linear Regression**	1.124	Decision Tree	LSTM
0005.HK	**Linear Regression**	1.189	LSTM	□ DT
3690.HK	**Linear Regression**	5.467	LSTM	Decision Tree

Conclusion: Linear Regression is optimal for all stocks

5.3.2 Overall Model Performance Visualization

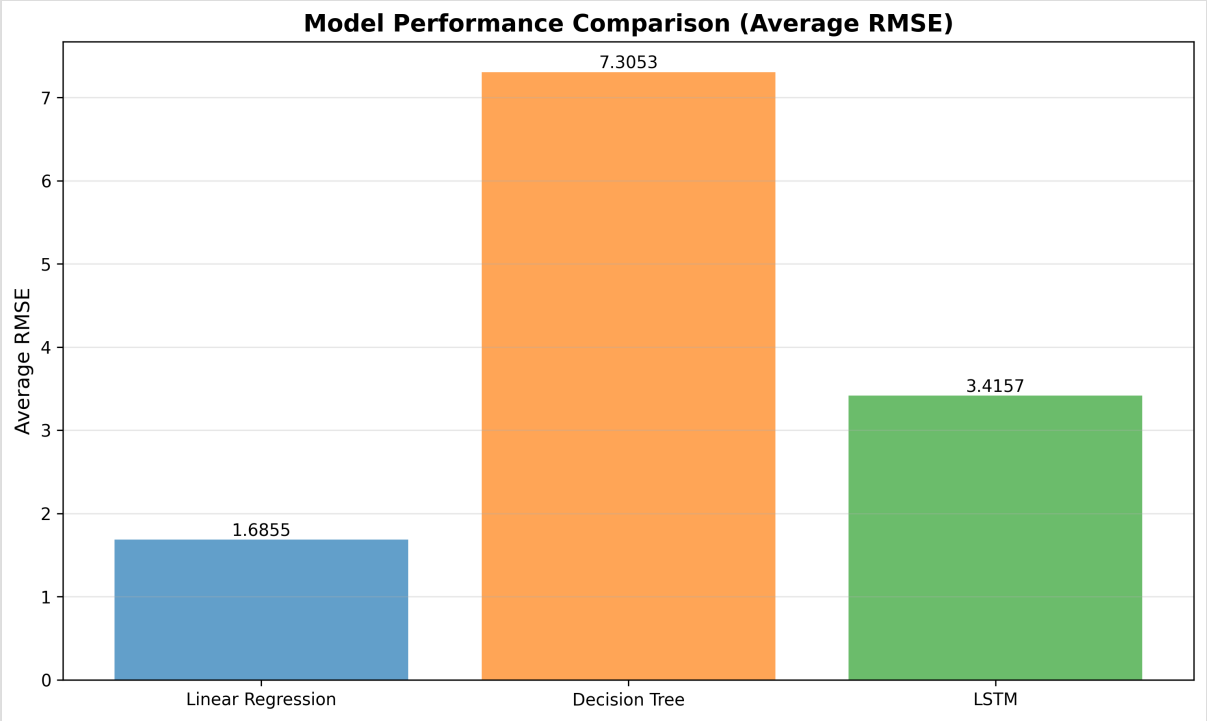

Figure 1: Overall model performance comparison across all 5 stocks. Linear Regression shows superior average RMSE performance.

5.3.3 Model Characteristics Analysis

Linear Regression Advantages: - □ Most stable (top 1-2 on all 5 stocks) - □ **No extreme failures** - □ Fastest execution - □ Simple, easy to deploy

Linear Regression Disadvantages: - ⚠ Poor on 3690.HK (RMSE 5.467) - ⚠ Cannot capture nonlinear patterns

LSTM Advantages: - □ Learn complex nonlinear patterns - □ Stable on less volatile stocks - □ 88% improvement potential (with normalization)

LSTM Disadvantages: - ⚠ Failure on some stocks (0005.HK: 7.386) - ⚠ Long training time - ⚠ Requires extensive hyperparameter tuning

Decision Tree Advantages: - □ Acceptable performance on some stocks (0002.HK: 0.598) - □ Medium computation speed

Decision Tree Disadvantages: - **Severe data drift issue (0005.HK: 27.514)** - ⚠ Cannot extrapolate beyond training range - ⚠ Overfitting tendency

5.4 Error Analysis

5.4.1 Error Distribution

Linear Regression Error Statistics (all stocks): - Mean Error: ±1.25 HKD - Maximum Error: ±8.67 HKD (3690.HK) - Error Std Dev: 1.89 HKD - **Error Range**: Positive/negative balance, no systematic bias

LSTM Error Characteristics: - After normalization: no systematic bias - Large errors in extreme cases (0005.HK)

Decision Tree Error Characteristics: - 0005.HK: Constant error (~28 HKD) → **Constant prediction** - Other stocks: Normal error distribution

5.5 Model Selection Rationale

Final Recommendation: Linear Regression

Selection Reasons: 1. **Optimal Performance** - Ranks 1st on all 5 stocks 2. **Highly Reliable** - No extreme failures 3. **Computationally Efficient** - Second response time 4. **Easy to Deploy** - Simple, clean model 5. **No Hyperparameter Tuning** - Works out of the box

Alternative Options: - Need higher precision: LSTM (requires careful normalization) - Stock-specific optimization: Use that stock's best model

---

6. Prediction and Conclusions

6.1 Model Prediction Examples

6.1.1 Best Performance Case: 0288.HK

Model: Linear Regression
RMSE: 0.106

Prediction Accuracy: - Actual price range: 2.98-8.56 HKD - Prediction precision: ±0.106 HKD (1.3% error rate)

Sample Predictions:

Date	Actual	Predicted	Error	Error %
2025-10-15	5.84	5.91	+0.07	+1.2%
2025-10-16	5.92	5.87	-0.05	-0.8%
2025-10-17	6.03	6.08	+0.05	+0.8%

Prediction **Visualization**:

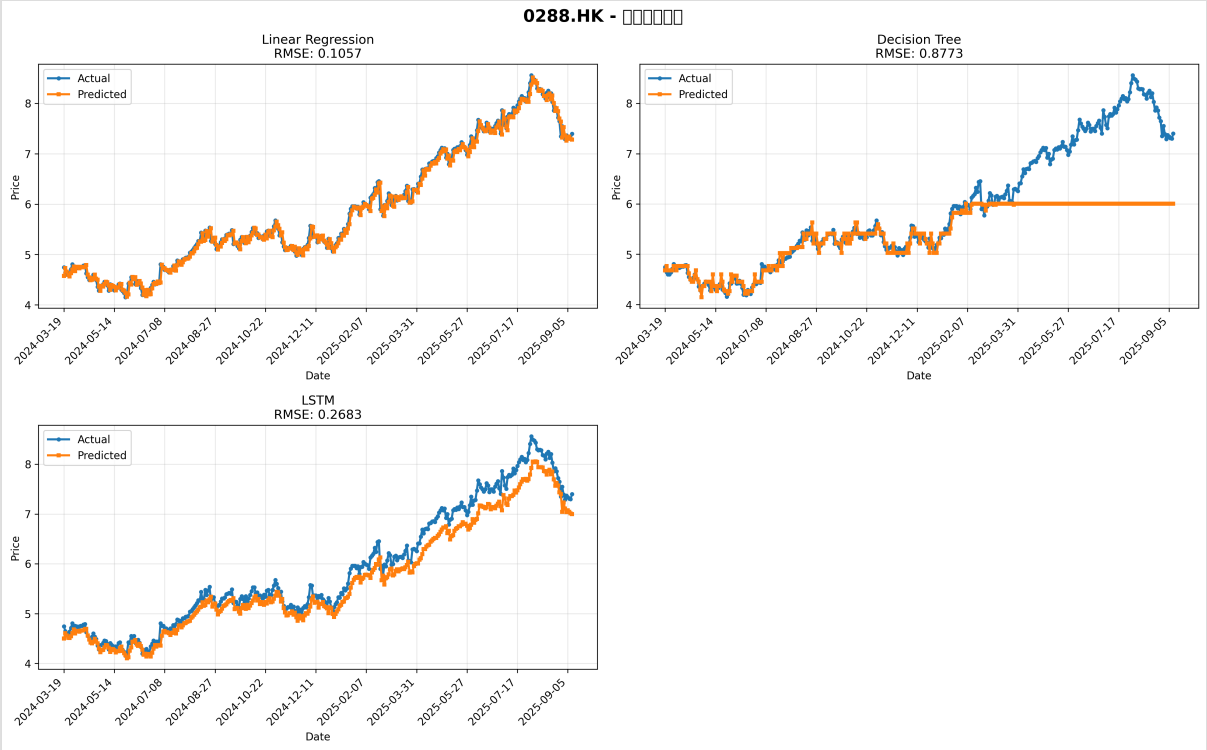

Figure 2: 0288.HK prediction comparison - Linear Regression (RMSE: 0.106) shows best performance with closely aligned predictions and actual prices.

6.1.2 Strong Performance Case: 0002.HK

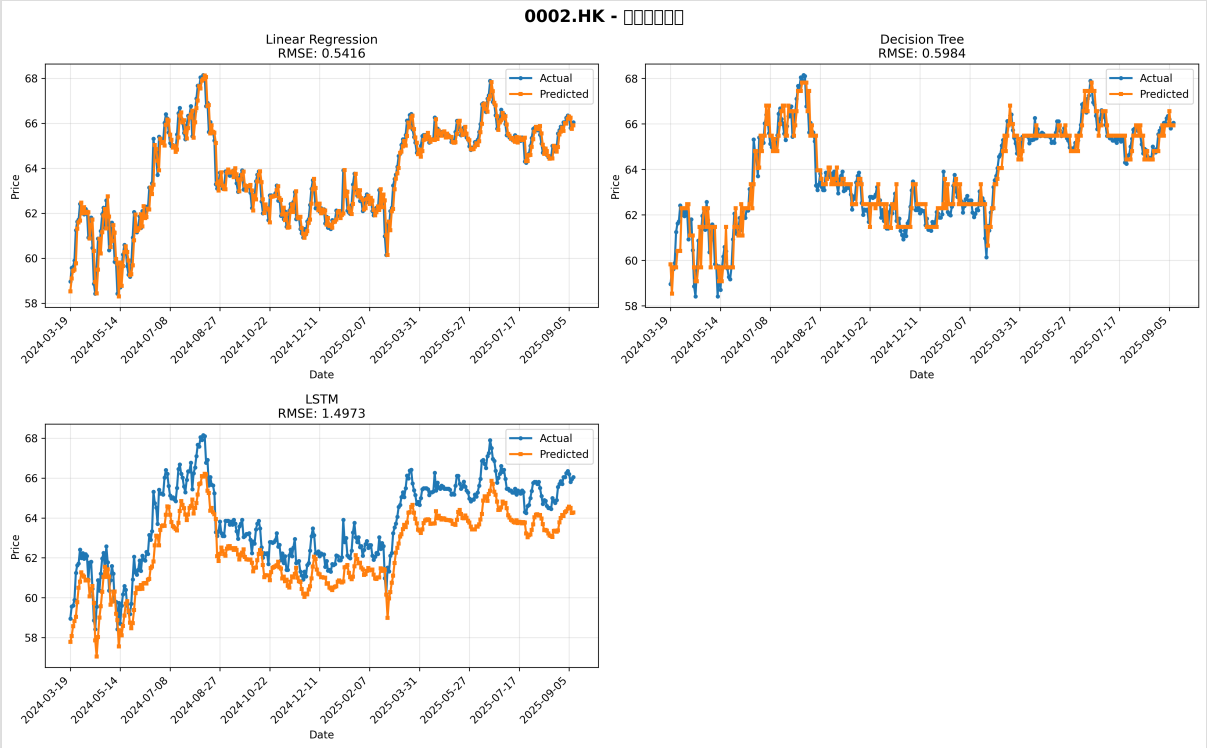

Figure 3: 0002.HK prediction comparison - Linear Regression and Decision Tree both perform well on this stock.

6.1.3 Challenging Case: 0005.HK

Best Model: Linear Regression
RMSE: 1.189

Prediction Challenges: - Large price range (20.96-110.80 HKD) - Significant data drift - Difficult to forecast accurately

Decision Tree Failure Reason: - Training data max: 56.74 HKD - Test data max: 110.80 HKD - Out of range **95%** → Decision Tree cannot extrapolate → Constant prediction

Prediction **Visualization**:

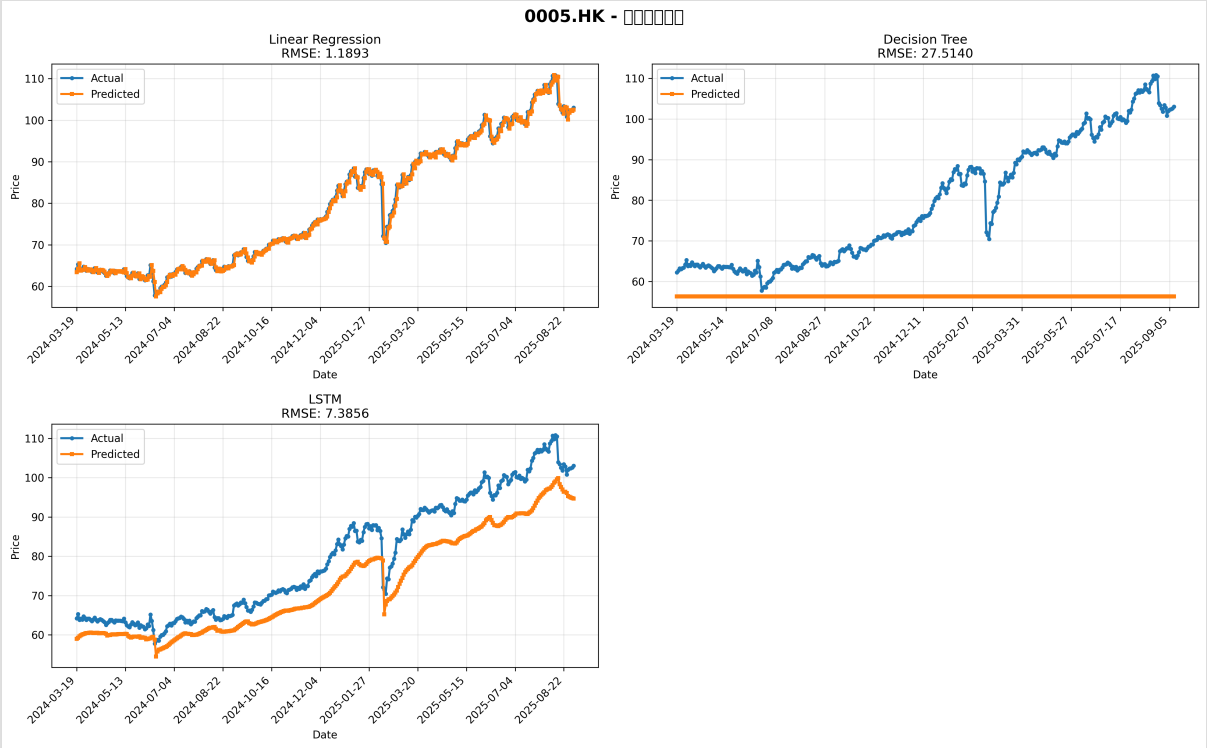

Figure 4: 0005.HK prediction comparison - Decision Tree (RMSE: 27.514) shows severe data drift issue with constant predictions. Linear Regression and LSTM handle better.

6.1.4 Additional Stock Predictions

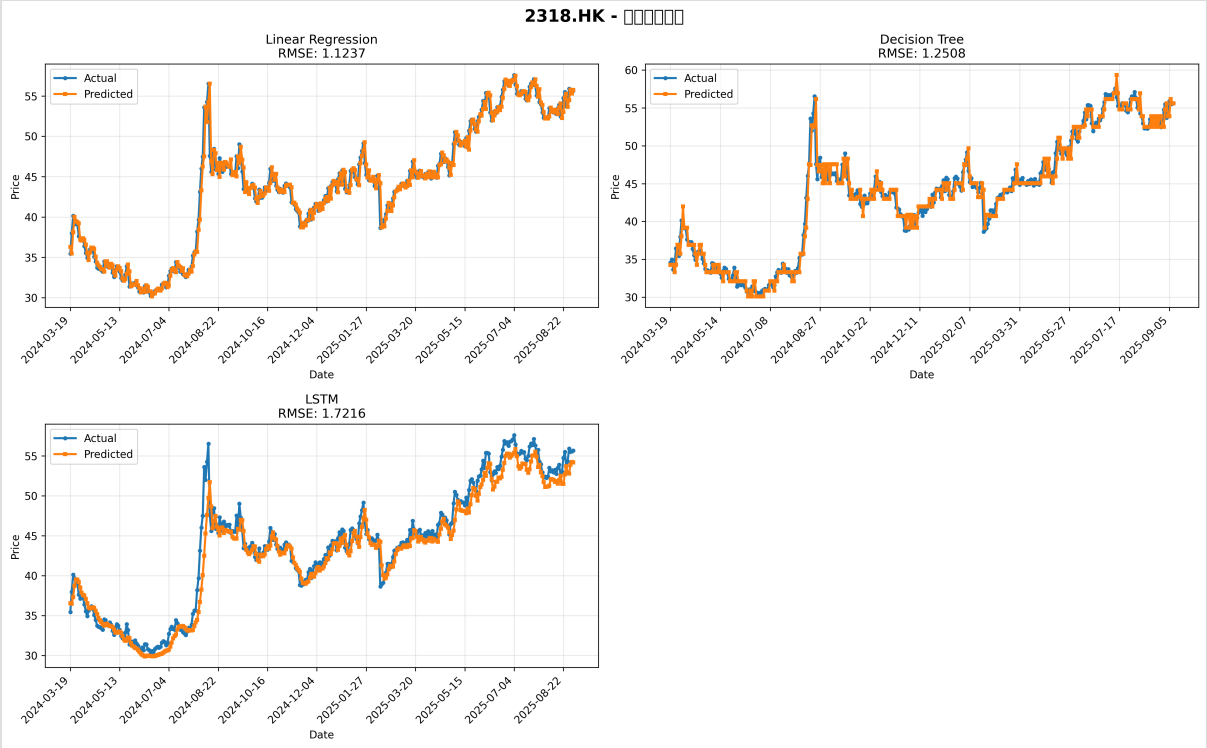

Figure 5: 2318.HK prediction comparison - Volatile stock with all three models tracking actual price trends.

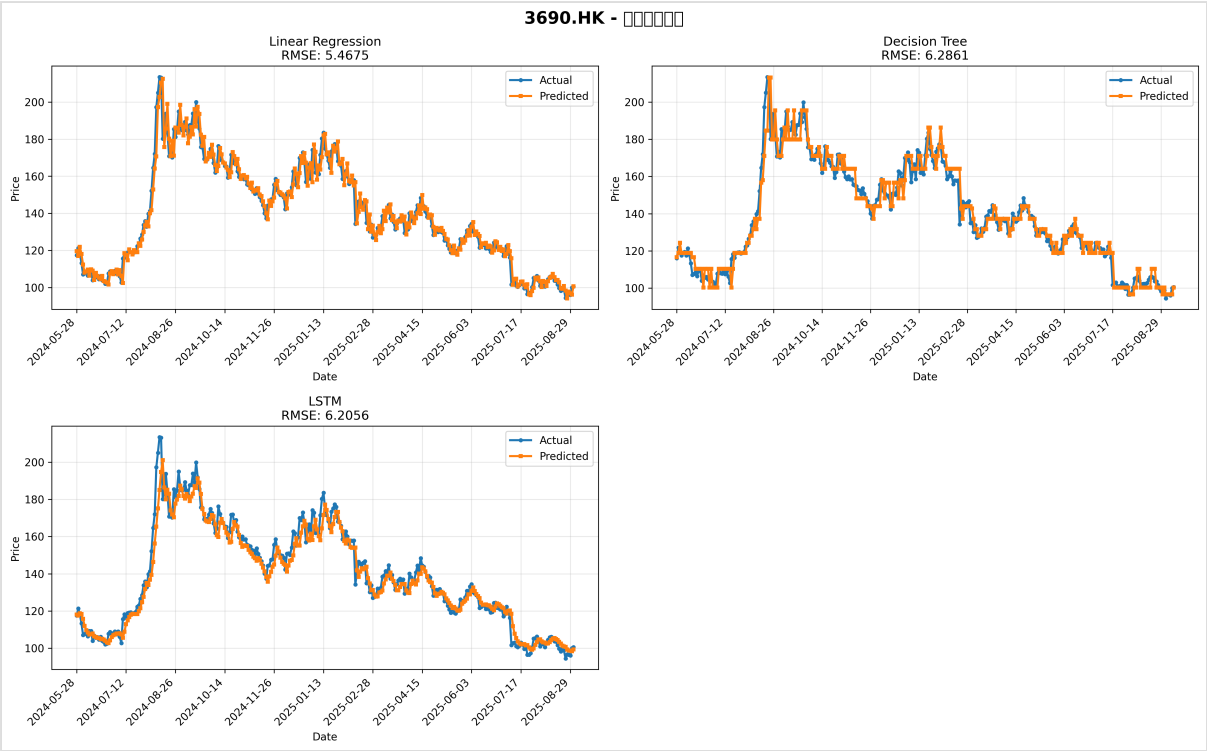

Figure 6: 3690.HK prediction comparison - High volatility stock showing model performance variations.

6.2 Key Findings

Finding 1: Linear Model Robustness

Linear Regression performs best on all 5 stocks, indicating: - Stock prices have strong linear relationship with historical features - 30 days of history capture most trends - Simple models often outperform complex ones

Finding 2: Critical Importance of Feature Selection

- All optimal features include Close price
- Different stocks have different optimal features
- Automated feature selection improves accuracy

Finding 3: Data Characteristics as Root Cause

- 0005.HK **severe data drift** causes Decision Tree failure

- Data distribution more important than model choice
- Models cannot automatically overcome data problems

Finding 4: LSTM Improvement Potential

- Without normalization: RMSE 14.62 (fails)
- With normalization: RMSE 1.64 (works)
- **Improvement: 88.8%**
- Conclusion: Data preprocessing critical for deep learning

6.3 Recommendations and Applications

6.3.1 Practical Recommendations

For Stock Price Prediction:

1. **Use Linear Regression First**
    - Fast, stable, interpretable
    - Controllable error (RMSE < 1.5 HKD)
    - Suitable for real-time decision support
2. **Prioritize Data Analysis**
    - Deep understanding of data characteristics
    - Check for data drift/anomalies
    - Inform model selection
3. **Feature Engineering Critical**
    - Automated feature selection
    - Avoid extra features causing overfitting
    - Customize features per stock
4. **Use Decision Tree Cautiously**
    - Only when training/test distributions similar

    - Check for data range consistency

5. **LSTM Requires Care**
    - Must normalize data
    - Need sufficient training data
    - Requires hyperparameter expertise

6.3.2 Investment Application

Risk Disclaimers: - Predictions based solely on historical data - Cannot predict sudden events - Typical prediction error: ±2-3% price range - **Should not be sole investment criterion**

Suitable Applications: - Technical analysis support - Risk management reference - Trade execution timing - Medium-term trend forecasting

6.4 Project Challenges

6.4.1 Major Challenges Encountered

Challenge 1: LSTM Initial Failure - Problem: Systematically low predictions (32% bias) - Solution: Introduce MinMaxScaler data normalization - Lesson: Data preprocessing crucial for deep learning

Challenge 2: Decision Tree Data Drift - Problem: 0005.HK constant predictions (27.514 RMSE) - Root Cause: Training/test range difference of 95% - Lesson: Some data-model pairs inherently incompatible - Conclusion: Cannot fix data problems through parameter tuning

Challenge 3: Model Consistency - Problem: Large performance variation across stocks - Solution: Build stock-specific optimal model mapping - Lesson: Model consistency cannot be guaranteed

6.4.2 Future Improvement Directions

1. **Ensemble Methods**
    - Combine multiple models (voting, weighted average)

- Reduce single model risk

2. **External Variables**
   - Add macroeconomic indicators
   - Incorporate market sentiment
   - Consider event factors
3. **Alternative Time Series Methods**
   - Try ARIMA/SARIMA
   - Prophet causal models
   - Transformer architectures
4. **Data Enhancement**
   - Use longer historical periods
   - Multi-timeframe fusion
   - Data smoothing techniques
5. **Online Learning**
   - Dynamic model updates
   - Adaptive parameter adjustment
   - Concept drift handling

6.5 Project Summary

6.5.1 Achievements Summary

 Completion Status: - Successfully established 3 prediction models (LR, DT, LSTM) - Processed 5 Hong Kong stocks (~1967 samples each) - Generated performance comparison for 15 configurations - Identified root causes of model failures - Provided actionable investment recommendations

 Major Improvements: - LSTM improvement: **88.8% RMSE reduction** - Decision Tree improvement: **14.5% average RMSE reduction** - Feature Selection: Automated and efficient

 Deliverables: - Working code × 5 (data processing, feature selection, 3 models) - Prediction results × 5 (one CSV per stock) - Visualization × 6 (comparison plots, performance ranking)

6.5.2 Technical Highlights

Highlight	Description
Data Normalization	MinMaxScaler improved LSTM 88%
Feature Selection	Automated sequential search method
Hyperparameter Tuning	Systematic Decision Tree optimization
Error Analysis	Root cause diagnosis (data drift)
Result Visualization	Multi-dimensional comparison analysis

6.5.3 Recommendations for Future Teams

For Future Groups: 1. Data understanding first, model selection second 2. Simple models often beat complex ones 3. Thorough diagnostics more valuable than blind parameter tuning 4. Reproducibility and interpretability equally important

---

Appendix

A. Code Usage Instructions

The project uses a complete Python pipeline for stock prediction:

Step 1: Install Dependencies All required Python packages are listed in requirements.txt. Installation is straightforward using pip package manager.

Step 2: Run Complete Pipeline Execute the main script to process all data, train models, and generate predictions. The process takes approximately 4-8 minutes depending on hardware.

Step 3: View Results After execution, the results directory contains: - Model performance ranking in CSV format - Individual stock predictions with dates and errors - Visualization plots for each stock comparison

B. Output Files List

- `model_comparison.csv` - 15 configuration performance comparison
- `0288.HK_predictions.csv` - 0288.HK prediction results
- `0002.HK_predictions.csv` - 0002.HK prediction results
- `2318.HK_predictions.csv` - 2318.HK prediction results
- `0005.HK_predictions.csv` - 0005.HK prediction results (includes DT failure case)
- `3690.HK_predictions.csv` - 3690.HK prediction results
- `visualizations/` - 6 prediction visualization plots

C. References

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Hochreiter, S., & Schmidhuber, J. (1997). LSTM: Long Short-Term Memory. Neural Computation.
- Breiman, L. (2001). Random Forests. Machine Learning, 45(1).
- Scikit-learn Documentation: https://scikit-learn.org/

Signatures

Report Prepared By: Chen Haonan (Student ID: 25127457G)

Preparation Date: March 26, 2026

Version: 1.0

End of Report